

Le patron de conception Pont

Durée : 1h00

Crédit : <https://www.math.univ-paris13.fr/~chaussar/>

1 Instructions générales

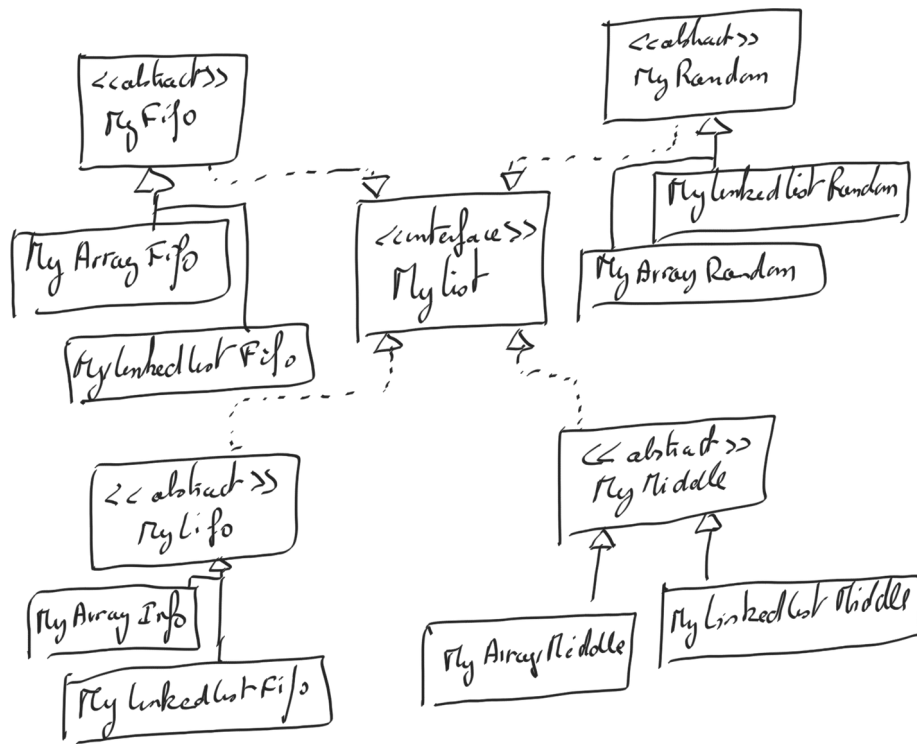
1. Téléchargez l'archive zip associée à cet énoncé sur la plateforme pédagogique.
2. Ouvrez le projet dans l'IDE IntelliJ IDEA (ou autre).
3. Si vous ne l'avez pas encore fait, configurez votre IDE pour n'accepter que des attributs avec un identifiant commençant pas un '_'. (*Dans IntelliJ IDEA, menu Settings::Editor::Code Style::Java, ajoutez le caractère '_' dans field et static field. Puis, dans le menu Settings::Editor::inspection::Java::Naming Conventions ajoutez la vérification de Field naming convention (instance field et static field) avec l'expression régulière : `_[a-z][A-Za-z\d]*`.*)
4. Exécutez le programme pour vérifier que tout fonctionne correctement.

2 Exercice 1

On souhaite réaliser le projet suivant : créer une interface `MyList` qui gère des ensembles d'entiers et qui contient trois fonctions : `push()`, `pop()`, et `isEmpty()`.

On ne considère ici que des listes d'objets quelconques (type `Object`) et donc des classes sans paramètre de type (ie. sans *template*). Cette interface pourra être déclinée en classes abstraites `MyLifo`, `MyFifo`, `MyRandom` (qui affiche un nombre choisi au hasard dans l'ensemble quand on appelle `pop()`) et `MyMiddle` (qui affiche le nombre rangé au centre de la liste quand on appelle `pop()`). Chacune de ces listes sera déclinée les classes `ArrayList` et `LinkedList` de Java.

Implémentez ces classes selon le diagramme de classes suivant et exécutez le `main()` fourni.



3 Exercice 2

On veut maintenant ajouter la gestion des ensembles avec des arbres binaires. Combien faut-il ajouter de classes pour implémenter cet ajout ?

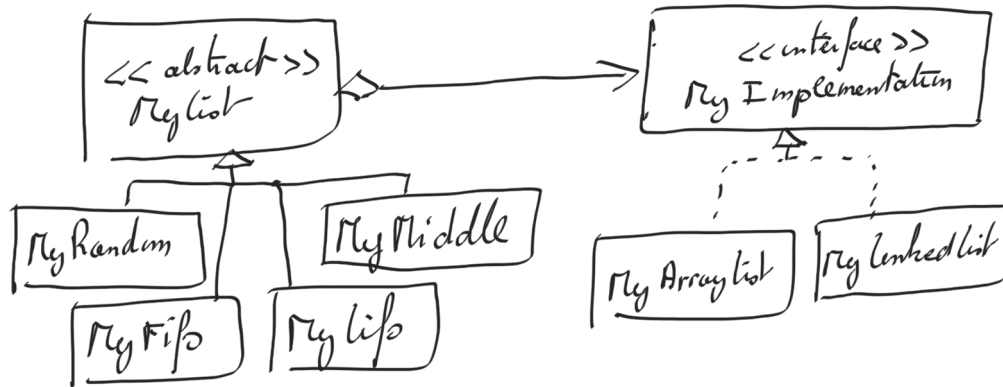
On veut aussi ajouter un système de gestion d'ensemble qui affiche à chaque fois le plus petit élément de la liste. Combien faut-il ajouter de classes pour implémenter cet ajout ?

Vous constatez donc que si vous avez N abstractions (type d'ensemble) et M implémentations (type de parcours), le jour où vous souhaitez rajouter une seule implémentation, vous devrez écrire N classes, et le jour où vous souhaitez rajouter une seule abstraction, vous devrez rajouter M classes.

On va donc séparer les abstractions des implémentations grâce au patron de conception Pont.

Vous devrez implémenter vos classes sans utiliser les classes `List` de Java (pas de `ArrayList`, `LinkedList`...). Vous aurez besoin d'une interface `MyImplementation`, qui contiendra les méthodes que vous choisirez (du genre `getSize`, `removeElementAt`, `addElementAt`) et dont hériteront les classes `ArrayList` et `LinkedList`. Vous aurez aussi besoin d'une classe abstraite `MyList`, qui contiendra un `MyImplementation`, des fonctions abstraites `pop`, `push` et `isEmpty`, et dont hériteront les classes `MyFifo`, `MyLifo`, `MyRandom` et `MyMiddle`.

Votre schéma de classe devra ressembler à ceci :



En séparant les systèmes d'accès aux éléments (`MyLifo`, `MyFifo`...) et les structures internes des listes (`MyArrayList`, `MyLinkedList`), on évite l'explosion de classe lorsque l'on souhaitera, plus tard, rajouter un élément. Pour s'en convaincre, combien de classe doit-on écrire pour ajouter les arbres binaires ? Combien de classes doit-on écrire pour ajouter l'implémentation qui retourne l'élément minimum d'un ensemble ?